Contromisura stutturale per evitare vulnerabilità

Description

La **programmazione robusta** è un approccio volto a minimizzare le vulnerabilità nei sistemi software, prevenendo gli effetti delle vulnerabilità e assicurando che il software funzioni correttamente anche in condizioni impreviste.

La robustezza del software è cruciale in contesti come la sicurezza informatica, dove i bug o le debolezze nel codice possono essere sfruttati da attaccanti per compromettere il sistema.

Ecco un'analisi dettagliata dei principi di programmazione robusta e delle regole di crittografia che dovresti seguire per scrivere codice sicuro e robusto:

1. Validare gli input del programma ("input is evil")

- **Validazione rigorosa** degli input è uno degli aspetti più importanti per evitare vulnerabilità. Gli input non validati possono essere sfruttati per causare **buffer overflow**, **iniezioni di codice** e altre vulnerabilità. È essenziale:
  - Verificare che i dati ricevuti siano **del tipo corretto** (ad esempio, numeri, stringhe, etc.).
  - Assicurarsi che non ci siano **dati malformati** che potrebbero causare comportamenti imprevedibili.
  - Usare whitelist per i caratteri accettati e **sanitizzare** gli input, rimuovendo caratteri speciali o potenzialmente pericolosi (ad esempio, `;`, `&`, `'`, `"`, `<`, `>` in contesti di SQL o HTML).
  - Gestire correttamente gli **input esterni** come URL, dati di rete, e file, assicurandosi che siano sicuri.

2. Impedire i buffer overflow

- **Buffer overflow** è una vulnerabilità che si verifica quando un programma scrive più dati di quelli che un buffer può contenere, sovrascrivendo dati

adiacenti in memoria. Questo può portare all'esecuzione di codice arbitrario.

- Per evitarlo:
  - Usa **funzioni sicure** per l'allocazione e la gestione della memoria, come `strncpy()` invece di `strcpy()`.
  - **Controlla sempre i limiti** di memoria prima di scrivere in un buffer.
  - Usa tecniche come la **protezione del stack**, che impedisce la modifica delle variabili di controllo del flusso del programma (ad esempio, protezione del ritorno del puntatore del frame).

3. Implementazione robusta limitando le informazioni trasmesse all'esterno di un modulo, oggetto, funzione

- Ogni modulo, funzione o oggetto dovrebbe avere un **interfaccia chiara e limitata**. Limitare le informazioni esposte ai vari componenti del programma riduce la superficie di attacco e la possibilità di accesso ai dati sensibili.
- **Puntatori logici** (riferimenti astratti agli oggetti) dovrebbero essere utilizzati anziché **puntatori fisici** (indirizzi di memoria concreti). I puntatori logici offrono maggiore **sicurezza** e protezione da manipolazioni accidentali o malevole.
- **Controllo delle informazioni trasmesse**: assicurati che solo le informazioni necessarie vengano passate tra le funzioni o tra i moduli del programma. La **principale regola** qui è evitare di esporre dati sensibili o internamente riservati al di fuori del contesto necessario.

4. Controllare i valori trasmessi ad altre funzioni (egress filtering)

- Quando un programma invia dati a un'altra funzione, è fondamentale **verificare e filtrare i valori** per evitare l'esecuzione di operazioni indesiderate. Questo è noto come **"egress filtering"**.
  - Ad esempio, se un programma invia un comando a un sistema esterno, è importante assicurarsi che il valore inviato non contenga codice potenzialmente dannoso (ad esempio, iniezioni di comandi o file path manipolati).

- L'uso di **funzioni di validazione** prima di inviare qualsiasi dato ad un'altra funzione è fondamentale per la sicurezza.

5. Controllare i risultati ricevuti

- Un altro passo cruciale è il controllo dei **risultati ricevuti** da funzioni esterne o da altre parti del programma.
  - Ad esempio, se una funzione restituisce un puntatore, è essenziale verificare se è `NULL` prima di utilizzarlo per evitare dereferenziazioni di puntatori nulli, che possono causare **crash** o vulnerabilità di esecuzione di codice malevolo.
  - Analogamente, è importante verificare che i dati restituiti da una funzione siano **corretti** e non contengano valori anomali o dannosi.

Errori da evitare (CWE Top 25 Most Dangerous Software Weaknesses)

Il **CWE Top 25** è una lista delle 25 vulnerabilità più comuni e pericolose nel software. Alcuni errori da evitare includono:

1. **Input validation errors**: Input non controllati che possono essere sfruttati per SQL injection, buffer overflow, e simili.
2. **Improper authentication**: Gestione debole delle credenziali degli utenti, che può portare a **elevazione dei privilegi** o **bypass della sicurezza**.
3. **Buffer overflow**: Errori nella gestione dei buffer che possono compromettere la memoria e il controllo del programma.
4. **Cryptographic issues**: Uso scorretto di algoritmi crittografici, come l'uso di chiavi troppo corte o la gestione scorretta dei segreti.

Crypto Coding: Regole per la crittografia

La **sicurezza crittografica** è un elemento fondamentale della programmazione sicura. Le regole che vanno seguite nella codifica della crittografia includono:

1. **Confronta le stringhe segrete in tempo costante**

- Quando confronti stringhe segrete (come password o chiavi), assicurati che il confronto avvenga in **tempo costante** per evitare attacchi di **tempo di esecuzione** (come gli attacchi di **timing attacks**).

2. **Evitare ramificazioni controllate da dati segreti**
   - Non usare ramificazioni condizionali che dipendono da dati segreti (come `if (secret == 0)`), poiché possono rivelare informazioni sui dati segreti a un attaccante.
3. **Evitare lookup di tabelle indicizzati da dati segreti**
   - Evita di usare tabelle o array indicizzati da dati segreti, poiché anche piccoli cambiamenti nei dati segreti possono causare una variazione facilmente misurabile nel comportamento del programma.
4. **Prevenire interferenze del compilatore con operazioni critiche per la sicurezza**
   - Assicurati che il compilatore non ottimizzi in modo pericoloso le operazioni critiche per la sicurezza, come la cancellazione di variabili contenenti dati segreti dalla memoria.
5. **Fornire API di alto livello**
   - Usa **API ad alto livello** che nascondano i dettagli di implementazione crittografica complessi. Le API di basso livello dovrebbero essere usate solo quando è strettamente necessario e solo con un'accurata gestione della sicurezza.
6. **Quando possibile, evita le API di basso livello**
   - Le **API di basso livello** possono essere più soggette a errori e complessità. Usa **API di alto livello** per gestire la crittografia in modo più sicuro e meno incline agli errori.
7. **Chiarire la distinzione tra API sicure e insicure**
   - Assicurati che le API sicure siano chiaramente separate da quelle insicure, in modo che gli sviluppatori non confondano quali funzioni possono essere usate in contesti di sicurezza.
8. **Usa byte senza segno per rappresentare i dati binari**

- Usa **unsigned bytes** quando rappresenti dati binari, per evitare confusione con i valori negativi che possono causare errori.

9. **Sempre eseguire il typecast dei valori spostati**
    - Quando si manipolano valori binari, esegui sempre il **typecast** dei valori spostati per evitare incongruenze o errori nei calcoli.
10. **Pulisci la memoria dei dati segreti**
    - Dopo aver usato dati segreti (come chiavi o password), **pulisci sempre la memoria** per impedire che possano essere recuperati in seguito.
11. **Usa randomizzazione forte**
    - Usa **generatori di numeri casuali sicuri** (ad esempio, basati su hardware o su algoritmi robusti) per garantire che i dati segreti generati siano realmente imprevedibili e sicuri.